

Regression Testing as the Operational Application of the Determinism Contract in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize regression testing — the operational mechanism by which the determinism contract's guarantees become verifiable in deployed systems — as a standalone architectural commitment with independent operational content, separable from the substrate/model distinction, the five guarantees themselves, and the categories of allowed non-determinism with which it composes.

Abstract

The CKS pattern's determinism contract commits a substrate to five guarantees about reproducibility and bounds the non-determinism the architecture permits at the model and operational layers. Companion decomposition notes formalize the substrate/model boundary, the five guarantees individually, and the allowed non-determinism. What remains to formalize is the operational verification mechanism: without it, the contract is aspirational — guarantees declared but not checkable. This note formalizes regression testing as having independent architectural content. The commitment is to operational feasibility — substrate state can be fixed, orchestration rules can be fixed, operations can be executed, and the guarantees can be verified — distinct from the deployment-level practice of running specific tests, maintaining test suites, or integrating tests into pipelines. The note states the four operational components the architecture must satisfy, specifies how the test mechanism accommodates the five categories of allowed non-determinism, distinguishes the commitment from four adjacent verification patterns commonly conflated with it, names ten failure modes that violate it, and provides an operational test for whether a given system's regression-testing architecture satisfies the commitment.

1. Why regression testing needs to be formalized as standalone

The parent foundational note formalizes the determinism contract — five guarantees about reproducibility, plus bounded non-determinism at the model and operational layers. Companion decomposition notes formalize the substrate/model distinction, the five guarantees individually, and the five categories of allowed non-determinism. What none of those notes does — and what this note does

— is treat regression testing as a commitment with independent architectural content, separable from the contract's other operational pieces.

Without an explicit standalone commitment, the determinism contract is aspirational. Five guarantees can be declared without being checkable; allowed non-determinism can be misattributed without being challengeable; the substrate/model boundary can be drawn without being verifiable in any specific deployment. The contract becomes enforceable rather than merely declared exactly when regression testing is committed to as the verification mechanism the architecture must support.

The motivating cases are the ones in which the contract has to do real work — governance reviews verifying substrate properties under specified conditions, audits comparing pre- and post-change substrate behavior under fixed inputs, compliance verifications confirming substrate determinism for regulated deployments. Each requires the substrate be in a state where its properties can be fixed and read against the contract's guarantees. Path retraceability per §3.1 supplies the substrate-internal reconstruction the test mechanism reads against; the human-governed commitment supplies the authority the test mechanism operationalizes; together with this commitment, those neighbors specify a substrate that is not merely reproducible in principle but verifiably so in practice.

2. The regression-testing commitment, defined precisely

The architectural commitment is to four operational components. A system that satisfies all four supports regression testing for the determinism contract in the architectural sense; a system that fails any one does not, regardless of what other testing capabilities it offers.

(a) Substrate-state fixing. The architecture supports fixing substrate state for the duration of a test. The test specifies a substrate state *S* as input, and the test executes against *S*. State-fixing may operate through any of several mechanisms — snapshotting a production substrate, constructing a deliberately-shaped test substrate, restoring a recorded state — and the architectural commitment is to state-fixing being feasible, not to any particular mechanism. What it forecloses is substrates whose state cannot be replicated, snapshotted, or restored for test purposes.

(b) Rule-set fixing. The architecture supports fixing the orchestration rule set for the duration of a test. Orchestration rules are substrate content per §2.1; fixing the rule set means fixing the substrate's rule content at the time the test runs. The commitment is to rule-fixing being feasible, not to any particular versioning or test-infrastructure technology. What it forecloses is implementations that mutate rules at runtime in ways that defeat fixing — hot-reload that changes rules underneath an executing test, runtime rule-injection by the LLM, or other dynamics that prevent the test from operating under a specified rule.

(c) Operation execution. The architecture supports executing specified operations or queries against the fixed substrate state and rule set, through the architecture's standard mechanisms — reads, cell executions, writes, conflict-related operations. The test does not bypass the architecture: it does not run cells outside the orchestration rules, write to substrate outside the standard write path, or skip boundary crossings between substrate and cell. Operations executed under test are operations of the kind the architecture supports in production, not synthetic operations through a test-only side channel.

(d) Guarantee verification. The architecture supports verifying that operation results conform to the guarantees. Verification is operationally specific to each guarantee: that reads return identical content for identical state; that cell behavior under fixed state and rules produces rule-equivalent substrate writes; that changes from operations are addressable as substrate content with provenance; that conflict states are preserved across operations; and that the contract operates substrate-internally without external-system overrides. The commitment is to substrate properties being readable in forms that permit verification — not to any particular verification framework or assertion language.

The four components are conjunctive. Failure of any one disables the test mechanism, because each names a capability the architecture must support for the verification to function at all.

3. How regression testing accommodates allowed non-determinism

The companion note formalizing allowed non-determinism enumerates five categories the architecture permits without violating the contract. Regression testing in CKS accommodates all five — verifying the contract's guarantees without imposing determinism the architecture does not commit to.

For LLM token output non-determinism, tests do not verify bit-identical model outputs. A test that fixes substrate state and a rule, then executes a cell, may observe different specific text content across runs; the test verifies rule-conformance — that the substrate writes the cell produces are correctly structured, populated, and constrained — not that the writes are character-identical.

For cell execution timing non-determinism, tests do not require cells to execute in specific orders or at specific moments. The test verifies substrate state at completion conforms to expected post-execution properties, regardless of the timing path that produced it.

For inter-cell communication timing non-determinism, tests do not require substrate-mediated communication between cells to occur on a specific schedule. The test verifies substrate state at communication boundaries — what each cell read, what each cell wrote — rather than the timing of the reads and writes.

For external-system response non-determinism, tests do not require external-system responses to be deterministic. For deployments that compose with adjacent components, the test may mock external responses, fix external-response inputs for the duration of the test, or accept varying external responses while verifying that cell-write rule-conformance holds regardless of the external input.

For hardware and infrastructure non-determinism, tests operate at the architectural-content level. The test verifies substrate content properties regardless of underlying infrastructure variation; the architectural commitment is to content-level verification consistent with tool-agnosticism's three minimal requirements, not to infrastructure-level reproducibility.

The five accommodations together specify what regression testing in CKS does not demand: bit-identical replay, deterministic timing, deterministic external responses, or deterministic infrastructure. Tests verify rule-conformance and substrate-state properties, which is what the contract binds.

4. What regression testing does NOT claim

The standalone treatment is bounded, and stating what it does not require keeps the commitment from drifting into something stronger than the source paper supports. It does not claim that all tests are bit-identical at the output level — tests verify rule-conformance and substrate-state properties, and specific bit-level outputs may vary within rule-allowed bounds. It does not require any specific test-infrastructure technology — implementations may use any combination of test frameworks, snapshot mechanisms, mocking patterns, or assertion libraries the deployment finds useful. It does not specify test-coverage policies — minimum percentages of substrate scopes tested or rules exercised are deployment concerns. It does not require continuous-integration or specific deployment-pipeline integration — tests may be run at deployment time, on demand, or in CI pipelines, as the deployment chooses. It does not foreclose other testing approaches — unit testing, integration testing, end-to-end testing, and property-based testing may be used alongside regression testing, the architectural commitment being to regression testing's operational feasibility for the determinism contract specifically. It does not require any specific test-result-format convention — results may be recorded, reported, or analyzed in any format the deployment chooses.

5. What regression testing is NOT

Four adjacent verification patterns are commonly conflated with regression testing in the CKS sense. Each is a real and reasonable commitment in some other architecture; naming what regression testing here is not is what prevents the misreading.

Not unit testing. Unit testing typically verifies individual code units in isolation, with mocked dependencies. Regression testing in CKS operates at the architectural-content level — it verifies substrate behavior under specified state and rules, not individual code units in isolation. A system may have comprehensive unit testing while failing the architectural regression-testing commitment if the unit tests do not verify the determinism guarantees at the substrate level.

Not integration testing. Integration testing typically verifies interactions between system components. Regression testing in CKS verifies substrate behavior under fixed inputs across the determinism guarantees. A system may have integration testing for component interactions while failing the regression-testing commitment, because component-interaction verification is not substrate-state-and-rule-conformance verification.

Not replay-based testing. Replay-based testing typically replays recorded executions and verifies reproducibility at the execution-trace level. Regression testing in CKS does not require execution-level replay; the allowed non-determinism the architecture permits makes execution-level replay inappropriate. The verification is at the rule-conformance and substrate-state level, not at the execution-trace level.

Not property-based testing. Property-based testing typically generates inputs and verifies properties hold across input space. Regression testing in CKS may use property-based approaches as a complement, but the architectural commitment is to specific verification of the guarantees against fixed substrate state and rules — different from property-based testing's focus on property-space exploration.

The two compose; they are not the same commitment.

The four distinctions together preserve the standalone treatment: regression testing in CKS names a specific verification commitment over the determinism contract, distinct from the adjacent patterns above and substitutable for none of them.

6. Why regression testing is load-bearing for downstream commitments

Regression testing is load-bearing for several CKS commitments beyond the determinism contract itself. Without it, the integrating contract is aspirational, the guarantees become unenforceable in any specific deployment, and the architectural difference the pattern claims against adjacent designs becomes unverifiable. The human-governed commitment per §2.1 grants humans the inspect, modify, and override rights at all times; regression testing operationalizes the inspect right's verification capability, turning it from a right held in principle into a capability exercised in practice. The path-retraceability commitment per §3.1 requires substrate paths be reconstructable from substrate content alone; regression testing exercises this reconstruction by traversing substrate paths to verify state, behavior, and change history at specified points. The substrate-as-source-of-truth commitment per §11.3 commits the substrate to authoritative state; regression testing verifies that authority operationally rather than merely architecturally. The architectural differentiation the source paper draws against adjacent designs — most pointedly in §5.2 against OIDA and in §6.2 against parametric-memory and external-structured-memory alternatives — manifests as substrate-level properties (authority as substrate content, conflict-as-first-class, substrate-only paths, tool-agnosticism), each of which is verifiable through regression testing under this commitment. The connection runs in both directions: regression testing depends on the other commitments for its scope, and the other commitments depend on regression testing for their operational meaning.

7. Failure modes that violate the operational verification commitment

A system can fail the regression-testing commitment specifically, even when it satisfies other CKS commitments and even when it has comprehensive testing capabilities for other purposes. Ten failure modes name the most common ways this happens.

(a) Substrate state cannot be fixed. The implementation does not support fixing substrate state — substrate is "always live" and cannot be replicated, snapshotted, or restored. Component (a) of §2 fails.

(b) Rule set cannot be fixed. Rules may change at runtime under hot-reload, runtime rule-injection by the LLM, or other dynamic mechanisms that prevent verifying cell behavior under a specified rule. Component (b) fails.

(c) Operations bypass the architecture during testing. The implementation supports test execution through "test hooks" or "bypass mechanisms" that skip standard boundary crossings or write paths. Tests verify behavior that does not match production execution paths. Component (c) fails.

(d) Guarantees cannot be verified. The implementation does not expose substrate properties — content, state, conflict records, change history — in forms that permit guarantee verification.

Component (d) fails.

(e) Tests require LLM determinism. The implementation requires LLM outputs to be bit-identical for tests to pass, which is operationally infeasible for current model technology. Incompatible with the LLM-output category of allowed non-determinism.

(f) Tests require infrastructure determinism. The implementation requires specific hardware, network, or storage configurations for tests to pass. Incompatible with the infrastructure category of allowed non-determinism and with tool-agnosticism.

(g) Tests cannot accommodate external-system non-determinism. The implementation requires external-system responses to be specific or deterministic for tests to pass. Incompatible with the external-response category of allowed non-determinism.

(h) Tests verify implementation details rather than guarantees. Tests assert against specific code paths, output formats, or intermediate states rather than against the architectural guarantees. Tests pass for one implementation and fail for an alternative implementation of the same architectural commitments — verification is brittle to implementation choice rather than verifying the architecture itself.

(i) Tests are not substrate-internal. Tests rely on external systems — audit logs from a separate platform, monitoring data from an external observer, state stores outside substrate — for the verification step. The substrate-as-source-of-truth guarantee is violated because verification depends on extra-substrate state.

(j) Tests are not migration-coherent. Tests cannot be run on a substrate after migration to an alternative host environment — the test mechanism depends on host-specific infrastructure beyond what tool-agnosticism's three minimal requirements supply. The test mechanism does not migrate with the substrate.

A system that exhibits any of (a)–(j) does not implement regression testing for the determinism contract in the architectural sense. The system may have other valuable testing capabilities, but those capabilities do not substitute for the commitment named here.

8. Operational test

A system supports regression testing for the determinism contract if and only if all of the following are true at all times during the substrate's existence:

1. Substrate state can be fixed for the duration of a test execution per component (a) of §2.
2. The orchestration rule set can be fixed for the duration of a test execution per component (b).
3. Operations can be executed against the fixed substrate state and rule set through the architecture's standard mechanisms — reads, cell executions, writes, conflict-related operations — without bypass paths, per component (c).

4. The five guarantees of the determinism contract can be verified through operation results, per component (d).
5. The test mechanism does not require LLM bit-identical outputs, deterministic cell timing, deterministic external-system responses, or deterministic infrastructure beyond the three minimal requirements of tool-agnosticism.
6. Verification depends on substrate's own properties, not on extra-substrate systems — the test mechanism is substrate-internal in the same sense the source-of-truth guarantee requires.
7. The test mechanism is migration-coherent — it can be run on a migrated substrate without depending on host-specific infrastructure beyond the three minimal requirements.

A system that fails any of (1)–(7) does not support regression testing for the determinism contract in the architectural sense, even if it has operational testing capabilities for other purposes.

9. Why naming regression testing as standalone matters

Implementations under pressure to deliver reliable AI-coordination behavior consistently drift in two directions. Some over-claim verification — presenting unit testing, integration testing, or generic quality-assurance practice as sufficient verification of the determinism contract, when those approaches verify at a different layer than the contract specifies. Others under-claim, treating verification as a deployment-level practice rather than as an architectural commitment, leaving the substrate's contract status unverifiable as a property of the architecture itself. The drift is steady because regression testing's specific scope — verifying the guarantees against fixed substrate state and rules with allowed non-determinism accommodation — is operationally distinct from conventional testing approaches and not obviously substitutable for them.

Implementations that drift away produce systems where the determinism contract is claimed but not verified. The downstream consequences are concrete: the contract cannot be enforced; humans exercising governance over the substrate cannot verify the properties they hold authority over; audits and compliance reviews cannot confirm substrate determinism; the architectural differentiation the pattern claims against adjacent designs cannot be defended in any specific instance. With this note in place, the eight-note decomposition of the determinism contract is complete: integrating frame, substrate/model boundary, the five guarantees, the allowed non-determinism, and now the operational verification mechanism. Subsequent work that implements, extends, or argues against the CKS determinism commitment should use "regression testing" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Regression Testing as the Operational Application of the Determinism Contract in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.